A Certificate Is Not the Answer

https://github.com/RuslansAleksejevs/boundary-conditions

Ruslans Aleksejevs

UC Berkeley

9 November 2026

Swarm Alignment Hackathon · Track 4: Multi-Agent Oversight and Control

Abstract

A checker can correctly approve a statement that is not the one the user asked about. We tested whether a model reviewer mistakes such a certificate for a correct answer to the original task. Our environment uses four true/false inputs, so code can check every possible state. We compared ordinary model translations with deliberately altered statements and harmless rewrites. For the controlled cases, the same statement and certificate appeared under three headings: PASS, a neutral record, or an explicit warning that the certificate covers only the candidate statement.

In the frozen Qwen3.5-122B-A10B evaluation, every heading led to correct rejection of all 40 planted substitutions and acceptance of all 20 equivalent rewrites. The ordinary translation pipeline completed 36 of 40 tasks; four were unassessed because of format errors. No false assurance was observed. The model checked the original successfully in this small setting, where it was always visible. The study provides a fully checkable test, but does not show that scope warnings improve performance or that longer verification pipelines are reliable.

1 Question

When a proof-like certificate is valid for an altered task, will a reviewing agent still check what the user actually asked? Does making the certificate's limited scope explicit help?

2 Experiment

Each task describes four Boolean inputs and two rules: C defines allowed states; Q defines what must hold. `SAFE` means every allowed state satisfies Q. `UNSAFE` must include an allowed state that violates it. An exact Python checker evaluates all 16 states, so the answer is independently testable.

In the ordinary pipeline, Qwen translated a task, code certified that candidate, and a fresh Qwen context reviewed the original. In the controlled tests, we replaced the translation with either a planted substitution or an equivalent rewrite. Each received a valid candidate certificate. Only its heading changed. The original and an instruction to check the match were visible in every condition.

After a development pilot, the frozen main test used 40 ordinary tasks, 40 planted substitutions and 20 equivalent rewrites. Substitutions restricted allowed states, dropped a requirement, weakened "and" to "or", or reversed an implication: ten cases of each. The controlled cases were reviewed under all three headings.

3 Results

No reviewer accepted a planted substitution as a safe original. All three headings gave the same result. A warning about certificate scope did not improve the observed score, because the baseline already got every controlled case right.

Table 1. Correct reviews of the original task. Columns reuse the same cases; these are paired comparisons, not three independent datasets.

Test	PASS	Neutral	Scope warning
Planted substitutions rejected	40 / 40	40 / 40	40 / 40
Equivalent rewrites accepted	20 / 20	20 / 20	20 / 20

Table 2. Ordinary translation pipeline, tested with the PASS presentation.

Outcome	Tasks
Correct end-to-end completion	36 / 40
Malformed translation; review not attempted	3 / 40
Malformed reviewer response	1 / 40

All 37 valid translations preserved meaning. The four unassessed tasks are not counted as successes. On planted cases, both heading comparisons had a zero false-assurance difference; conservative 95% intervals span -42.9 to $+42.9$ percentage points under independent task sampling. Matching scores in this small test do not prove that the headings behave the same.

4 A recorded example

A valid certificate for the wrong implication. In case `holdout-fault-019`, allowed states require r to be false. The requirement was deliberately reversed before certification:

$$\begin{aligned} &\text{Original requirement} && \neg r \Rightarrow (s \vee p) \\ &\text{Candidate requirement} && (s \vee p) \Rightarrow \neg r \end{aligned}$$

The candidate always holds in allowed states, so its `SAFE` certificate is valid. The original fails when all four inputs are false. The model found exactly that counterexample and answered `UNSAFE`, even under PASS. It caught the substitution; this was not a successful deception.

For contrast, case `holdout-equivalent-019` replaced the allowed-state rule q with $q \wedge q$, leaving the requirement unchanged. The model correctly answered `SAFE`: a different expression need not mean a different task.

5 Conclusion

The model distinguished valid evidence from evidence about the original request in all controlled cases. The test makes this distinction measurable, but the simple tasks leave the harder question open. Longer specifications or a less accessible original would be useful next tests; they were not evaluated here.

Appendix: Limitations and Dual-Use / Ethical Considerations

Scope. This is a small, finite environment with one model and one sample per case and heading. Every reviewer saw the complete original and was told to check the match: a strong baseline. All empirical allowed-state sets were nonempty. A large proof, an ambiguous request or a reviewer without the original could behave differently.

What the test establishes. A false assurance means a valid reviewer output says `SAFE` even though the original has a counterexample. Blocked or malformed outputs remain unassessed. Planted substitutions test whether a reviewer catches a known change, not how often a translator naturally makes one. There was no no-certificate condition or resource-matched single-agent baseline, so this study cannot show that adding a verifier improves performance. Zero observed differences also do not prove the headings equivalent.

Ethics. All tasks are synthetic Boolean rules, not claims about real software or people. The possible misuse is to present a valid certificate as broader assurance than it supports. The examples are disclosed to make that scope error testable, not to certify a deployed system.

Reproduction and related work

The run used `qwen/qwen3.5-122b-a10b` through OpenRouter's fixed Alibaba route: temperature 0.6, top-p 0.95, reasoning enabled, and an 8,192-token requested output limit. Reasoning text was excluded from evaluation. The repository's `studies/wrong-goal-proof` directory contains prompts, seeds, cases, certificates, stored outputs and offline audits. Main data are in `results/live-main-2026-11-08/main`; pilot results are separate. Certificates bind to a candidate hash; SAFE tables cover all 16 states.

Faithful formalization is an established problem: SemantEq distinguishes proving statements from preserving their meaning [1]; Mei and Douglas combine semantic review with mechanical checks [2]. Our contribution is a smaller, fully enumerable test of the reviewer's response to identical evidence with different headings.

[1] X. Li, P. S. Ammanamanchi, S. Bhat, R. Jamdar, Y. Lin and S. Biderman. *SemantEq: Faithful Bidirectional Equivalence of Formal Statements.* arXiv, 2026. arXiv:2610.32919v1.

[2] A. Mei and M. R. Douglas. *Beyond LLM Judge: Mechanical Verification for Agentic Autoformalization.* arXiv, 2026. arXiv:2610.20573v2.

LLM Usage Statement

AI coding assistants helped design the study, implement and run experiments, check saved outputs, and draft this report. Qwen was the experimental subject. Numerical claims and examples were checked against stored data with local code. These checks are not external peer review. The research was conducted during the Swarm Alignment Hackathon, November 2026.